

DEPARTMENT OF APPLIED MATHEMATICS

Project 1 (ODE IVP) Report

NUMERICAL SOLUTION OF THE ROBERTSON CHEMICAL KINETICS PROBLEM

GROUP E

Member	Role
Yi.Zhu (2361666)	A. Project Manager
Zhengdao.Song (2361511)	B. Mathematical Theory
Fangchi.Shi (2359837)	C. Algorithm Implementation
Xiangyu.Zhang (2361653)	D. Visualization & Report
Rong.Yin (2363466)	E. Testing & Validation

September 2026

Abstract

We study the Robertson problem, the three-species kinetics system proposed by H. H. Robertson in 1966 that has become the standard benchmark for stiff initial-value solvers: its rate constants span nearly nine orders of magnitude, so fast and slow modes coexist along a single trajectory and defeat naive explicit time stepping. We implemented three methods from scratch in Python: explicit Euler (EE), classical fourth-order Runge–Kutta (RK4), and implicit Euler (IE) with Newton iteration, the last wrapped in an adaptive step-doubling controller with an integral (I) control law. On a scalar nonstiff test equation the measured orders are 1.0, 4.0, and 1.0 for EE, RK4, and IE, matching theory. On the Robertson problem itself, EE and RK4 are stability-limited to $h \lesssim 5.9 \times 10^{-4}$ and $h \lesssim 8.2 \times 10^{-4}$, agreeing with frozen-Jacobian predictions to within 1–2%, while IE remains stable at $h = 0.25$. The adaptive IE solver spans six decades of step size, and its global error scales like $\sqrt{\text{rtol}}$ — a factor of about three per decade — the expected signature of an order-1 method under step-doubling control, for which the accepted step obeys $h \propto \text{rtol}^{1/2}$. In a work–precision comparison, SciPy’s Radau at $\text{rtol} = 10^{-6}$ attains weighted error 1.75×10^{-4} in 0.040 s, while our adaptive IE reaches only 205 in 0.65 s. The main limitation is the first-order accuracy ceiling of our implicit method combined with pure-Python overhead.

Keywords: stiff ODEs, Robertson problem, Runge–Kutta method, implicit Euler, Newton iteration, absolute stability, adaptive step-size control, step doubling, chemical kinetics

1. INTRODUCTION AND MOTIVATION

1.1 Problem Background

Chemical reaction networks almost never evolve on a single time scale: in combustion, atmospheric chemistry, and enzyme kinetics, some elementary reactions equilibrate in microseconds while the overall composition drifts over seconds or hours. Written as ODEs, the fast reactions appear as rapidly decaying modes of the linearised dynamics; once dead, they no longer influence the visible evolution, yet they still dictate the largest step any explicit integrator may take. Curtiss and Hirschfelder identified this phenomenon in 1952 and coined the term *stiff* [3]; Gear’s monograph [4] and the two volumes of Hairer and Wanner [5, 6] turned it into a mature theory.

The system we study is the canonical example. Robertson introduced it in 1966 as a model of an autocatalytic reaction [10], and it has served ever since as a standard benchmark: it appears in the test sets of the MATLAB ODE suite [12], of SUNDIALS [8], and of VODE [1], and in solver comparisons for atmospheric chemistry [11] and general reviews of stiff solvers [2]. Three components and three rate constants suffice to exhibit the full pathology of stiffness: the constants span $k_3/k_1 = 7.5 \times 10^8$, and the stiffness ratio along the solution grows from $\approx 3.0 \times 10^3$ at $t = 10^{-4}$ to 1.5840×10^5 at $t = 40$ (Figure 3). The project brief asks four things—the nonzero Jacobian eigenvalues and a stiffness ratio along the trajectory; the experimental stability/accuracy/positivity threshold of EE with a matched-error IE comparison; conservation, non-negativity, and full-state error as separate diagnostics; and a cross-check of the implicit method’s $y_2(40)$ against a self-constructed reference—and each is addressed in Section 4.

1.2 Mathematical Model

We consider the initial-value problem

$$\dot{y} = f(t, y), \quad y(0) = y_0, \quad t \in [0, T], \quad (1)$$

with $y(t) = (y_1, y_2, y_3)^\top \in \mathbb{R}^3$ the concentrations of three species $[A]$, $[B]$, $[C]$. The Robertson system is

$$\begin{aligned} \dot{y}_1 &= -0.04y_1 + 10^4 y_2 y_3, \\ \dot{y}_2 &= 0.04y_1 - 10^4 y_2 y_3 - 3 \times 10^7 y_2^2, \\ \dot{y}_3 &= 3 \times 10^7 y_2^2, \end{aligned} \quad y(0) = (1, 0, 0)^\top, \quad 0 \leq t \leq T = 40, \quad (2)$$

Reaction channels and time scales. Table 1 decomposes (2) into its three reaction channels. The rate constants span nearly nine orders of magnitude ($k_3/k_1 = 7.5 \times 10^8$), so slow and fast chemistry coexist along one trajectory—the physical origin of the stiffness studied throughout. The largest rate *constant* is not the largest *rate*: R3 is throttled by the intermediate ($y_2 \leq 3.7 \times 10^{-5}$ along the trajectory) and R2 by the product ($y_3 < 0.29$), so the largest Jacobian element encountered is $|J_{22}| = 10^4 y_3 + 6 \times 10^7 y_2 \approx 3.4 \times 10^3$, about four orders below the naive guess k_3 .

Table 1: The three reaction channels of the Robertson system. The rates of R2 and R3 are state-dependent; the values quoted are maxima along the reference trajectory of Section 4 and set the fast time scale.

Channel	Reaction	Rate constant	Largest rate sensitivity	Role in the dynamics
R1	$A \rightarrow B$	$k_1 = 0.04$	0.04	slow drift, time scale ≈ 25 s
R2	$B + C \rightarrow A + C$	$k_2 = 10^4$	$10^4 y_3 \approx 2.8 \times 10^3$ at $t = 40$	fast exchange; C catalytic
R3	$B + B \rightarrow B + C$	$k_3 = 3 \times 10^7$	$6 \times 10^7 y_2 \approx 2.2 \times 10^3$ at $t \approx 0.0046$	very fast bimolecular consumption of B

Conservation. Adding the three equations of (2) and regrouping by channel,

$$\dot{y}_1 + \dot{y}_2 + \dot{y}_3 = (-0.04y_1 + 0.04y_1) + (10^4 y_2 y_3 - 10^4 y_2 y_3) + (-3 \times 10^7 y_2^2 + 3 \times 10^7 y_2^2) = 0,$$

each pair corresponding to R1, R2, R3 in turn: every reaction merely redistributes matter among the species. Hence

$$(1, 1, 1)^T f(y) = 0 \quad \text{for all } y, \quad (3)$$

and integrating (3) from $y(0) = (1, 0, 0)^T$ confines the exact trajectory to the invariant plane $y_1 + y_2 + y_3 = 1$.

Positivity. On each coordinate face the vector field points back into the positive orthant,

$$y_1 = 0 \Rightarrow f_1 = 10^4 y_2 y_3 \geq 0, \quad y_2 = 0 \Rightarrow f_2 = 0.04 y_1 \geq 0, \quad y_3 = 0 \Rightarrow f_3 = 3 \times 10^7 y_2^2 \geq 0,$$

so a solution that starts non-negative stays non-negative. Conservation does not imply positivity—the state $(1.5, -0.6, 0.1)^T$ satisfies the invariant yet is physically meaningless—so Section 4 reports them as separate diagnostics: near its stability boundary EE conserves the total to machine precision while driving y_2 negative.

The Jacobian and the origin of stiffness. Differentiating (2) term by term, $J_{ij} = \partial f_i / \partial y_j$. The decisive entry is the response of B 's destruction rate to y_2 itself, through both fast channels:

$$J_{22} = \frac{\partial}{\partial y_2} (0.04 y_1 - 10^4 y_2 y_3 - 3 \times 10^7 y_2^2) = -10^4 y_3 - 6 \times 10^7 y_2.$$

The full Jacobian is

$$J(y) = \begin{pmatrix} -0.04 & 10^4 y_3 & 10^4 y_2 \\ 0.04 & -10^4 y_3 - 6 \times 10^7 y_2 & -10^4 y_2 \\ 0 & 6 \times 10^7 y_2 & 0 \end{pmatrix}. \quad (4)$$

At $y_2 = y_3 = 0$ every fast term vanishes and only the gentle -0.04 mode is active—stiffness is something the trajectory *grows into*, as C accumulates and the B transient passes.

A structural zero eigenvalue. Differentiating the identity (3) gives $(1, 1, 1)^T J(y) = 0$ for every y , and indeed every column of (4) sums to zero (e.g. column 2: $10^4 y_3 - (10^4 y_3 + 6 \times 10^7 y_2) + 6 \times 10^7 y_2 = 0$). The left null vector of J is the normal of the invariant plane, so J always has a zero eigenvalue—the algebraic shadow of conservation. Following the project brief, the stiffness ratio is built from the nonzero eigenvalues only,

$$S(t) = \frac{\max_{\lambda_j \neq 0} |\operatorname{Re} \lambda_j(t)|}{\min_{\lambda_j \neq 0} |\operatorname{Re} \lambda_j(t)|}, \quad (5)$$

classifying an eigenvalue as zero when $|\lambda_j| < 10^{-8}$; the reported stiffness ratios instead use the reduced Jacobian below, where the structural zero is excluded algebraically rather than by a threshold.

Degeneracy at $t = 0$. At the initial state the fast channels are inactive, so the second and third columns of $J(y_0)$ vanish and the spectrum is exact:

$$J(y_0) = \begin{pmatrix} -0.04 & 0 & 0 \\ 0.04 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}, \quad \det(J(y_0) - \lambda I) = -\lambda^2(\lambda + 0.04) = 0 \Rightarrow \sigma(J(y_0)) = \{-0.04, 0, 0\}.$$

The two zeros have different pedigrees: one is the structural eigenvalue of conservation, present at any state; the other an accidental degeneracy of the initial condition, splitting into a genuine nonzero eigenvalue as soon as y_2 turns positive. With only one active mode at $t = 0$ the ratio (5) is undefined there, so the plots of Section 4 begin at $t = 10^{-8}$.

The reduced Jacobian. Eliminating $y_3 = 1 - y_1 - y_2$ through the invariant reduces the right-hand side to

$$\tilde{f}_1 = -0.04y_1 + 10^4y_2(1 - y_1 - y_2), \quad \tilde{f}_2 = 0.04y_1 - 10^4y_2(1 - y_1 - y_2) - 3 \times 10^7y_2^2,$$

and differentiating with respect to (y_1, y_2) —for instance $\partial \tilde{f}_1 / \partial y_2 = 10^4(1 - y_1 - y_2) - 10^4y_2 = 10^4(y_3 - y_2)$ —gives

$$J_{\text{red}}(y_1, y_2) = \begin{pmatrix} -0.04 - 10^4y_2 & 10^4(y_3 - y_2) \\ 0.04 + 10^4y_2 & -10^4(y_3 - y_2) - 6 \times 10^7y_2 \end{pmatrix}, \quad y_3 = 1 - y_1 - y_2. \quad (6)$$

Because conservation confines the flow to the invariant plane, the two eigenvalues of J_{red} are exactly the two nonzero eigenvalues of (4); we confirmed this numerically at the sampled output times (agreement to $\sim 10^{-12}$, all real; a narrow complex pair $\lambda_{1,2} \approx -0.04 \pm 5.2 \times 10^{-4}i$ near $t \approx 1.7 \times 10^{-8}$ lies between samples). The stiffness data of Section 4 are computed from (6), so exclusion of the structural zero is built into the algebra rather than enforced by a threshold. Along a tight-tolerance reference the larger eigenvalue magnitude reaches $|\lambda|_{\max} = 3392.79$ at $t = 40$, and we reproduce the project-pack check values exactly: $S(10^{-4}) = 2996$, $S(10^{-2}) = 5423$, $S(40) = 1.5840 \times 10^5$.

1.3 Overview of the Report

Section 2 derives the three discrete schemes, their consistency, stability functions, and the convergence argument. Section 3 describes the implementation: two algorithms in pseudocode, the code structure, and key design decisions. Section 4 presents the numerical experiments, and Section 5 concludes.

2. DISCRETISATION AND ANALYSIS

We now turn the model (2) into discrete recursions and ask the three questions that decide whether a recursion is useful: is it consistent with the ODE, is it stable, and do the two together give convergence?

2.1 Discrete Schemes

Let $t_n = t_0 + nh$ with constant step $h > 0$ (the adaptive version in Section 3 relaxes this), and let $y_n \approx y(t_n)$.

Explicit Euler (EE). Replacing the derivative in (1) by a forward difference at t_n and solving for the unknown at t_{n+1} gives

$$y_{n+1} = y_n + hf(t_n, y_n). \quad (7)$$

The method is explicit: each step costs one evaluation of f at data already in hand, and no linear algebra.

Classical fourth-order Runge–Kutta (RK4). Sampling the vector field at four points inside the step matches the Taylor expansion of the exact solution through order four:

$$k_1 = f(t_n, y_n), \quad k_2 = f(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_1), \quad k_3 = f(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_2), \quad k_4 = f(t_n + h, y_n + hk_3), \quad (8)$$

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4). \quad (9)$$

The method is defined by its Butcher tableau (Table 2); since A is strictly lower triangular, every stage is computable from the previous ones, and the method is explicit with four evaluations of f per step.

Table 2: Butcher tableau of the classical RK4 method: nodes c (left column), stage couplings A (strictly lower-triangular block), quadrature weights b^T (bottom row).

c	A			
0				
$\frac{1}{2}$	$\frac{1}{2}$			
$\frac{1}{2}$	0	$\frac{1}{2}$		
1	0	0	1	
b^T	$\frac{1}{6}$	$\frac{1}{3}$	$\frac{1}{3}$	$\frac{1}{6}$

Implicit Euler (IE) with Newton iteration. Evaluating the vector field at the unknown end of the step instead,

$$y_{n+1} = y_n + hf(t_{n+1}, y_{n+1}), \quad (10)$$

defines y_{n+1} only implicitly: a nonlinear algebraic system at every step. We solve it with Newton's method applied to the residual

$$G(z) = z - y_n - hf(t_{n+1}, z) = 0, \quad (11)$$

whose derivative is $G'(z) = I - hJ(t_{n+1}, z)$ with J from (4). Starting from an initial guess $z^{(0)}$ (Section 3.3), the iteration

$$(I - hJ(t_{n+1}, z^{(k)})) \Delta z^{(k)} = -G(z^{(k)}), \quad z^{(k+1)} = z^{(k)} + \Delta z^{(k)} \quad (12)$$

runs until $\|G(z^{(k+1)})\|_\infty \leq \tau_N$ (a prescribed Newton tolerance) or a cap of 12 iterations rejects the step, which is then retried at $h/2$. Each iteration costs one evaluation of f , one of J , and one 3×3 LU solve; the algebraic error so introduced is a separate, controllable error source, quantified in Section 4.4. All three methods are one-step recursions $y_{n+1} = \Phi_h(y_n)$, the form that makes the analysis below uniform.

2.2 Consistency and Truncation Error

Assume the exact solution of (1) is as smooth as needed (f in (2) is polynomial, hence analytic). Taylor expansion about t_n gives

$$y(t_n + h) = y(t_n) + h\dot{y}(t_n) + \frac{h^2}{2}\ddot{y}(t_n) + \frac{h^3}{6}\dddot{y}(t_n) + O(h^4). \quad (13)$$

Define the local truncation error as the defect left when the exact solution is inserted into the scheme, $\tau_{n+1} = y(t_{n+1}) - \Phi_h(y(t_n))$.

EE. Subtracting (7) from (13) and using $\dot{y} = f$,

$$\tau_{n+1}^{\text{EE}} = \frac{h^2}{2} \ddot{y}(t_n) + O(h^3) = O(h^2), \quad (14)$$

so EE is consistent of order $p = 1$ (one power of h is absorbed by the $1/h$ in the defect-per-unit-step convention).

IE. Expanding instead backward from t_{n+1} , $y(t_n) = y(t_{n+1}) - h\dot{y}(t_{n+1}) + \frac{h^2}{2}\ddot{y}(t_{n+1}) + O(h^3)$, and inserting into (10),

$$\tau_{n+1}^{\text{IE}} = -\frac{h^2}{2} \ddot{y}(t_{n+1}) + O(h^3) = O(h^2), \quad (15)$$

so IE is also first order. The sign difference between (14) and (15) is visible in practice: on decaying modes EE undershoots and IE overshoots (for $y' = -y$, $h = 0.1$: $0.9 < e^{-0.1} \approx 0.9048 < 0.9091$). IE's robustness on stiff problems comes from its stability function, not this sign pattern (Section 2.3).

RK4. Matching (8)–(9) against (13) requires expanding all four stages and the elementary differentials of f up to order four; the algebra is standard [5, §II.1] and yields

$$\tau_{n+1}^{\text{RK4}} = O(h^5), \quad (16)$$

so RK4 is consistent of order $p = 4$. These orders hold for h small enough that the Taylor remainder is dominated by its leading term; on stiff problems there is a second, separate condition coming from stability—the subject of the next subsection, and exactly what makes the Robertson problem interesting.

2.3 Stability Analysis

Consistency says the scheme follows the ODE locally; stability asks whether the recursion amplifies the decaying modes that the ODE itself damps. The standard instrument is the Dahlquist test equation

$$\dot{y} = \lambda y, \quad \lambda \in \mathbb{C}, \quad \text{Re } \lambda < 0, \quad (17)$$

whose exact solution decays. Every one-step method applied to (17) reduces to

$$y_{n+1} = R(z) y_n, \quad z = h\lambda, \quad (18)$$

with a method-specific stability function $R(z)$; the absolute-stability region is $\mathcal{S} = \{z \in \mathbb{C} : |R(z)| \leq 1\}$. Direct substitution into (7), (8)–(9), and (10) gives

$$R_{\text{EE}}(z) = 1 + z, \quad (19)$$

$$R_{\text{RK4}}(z) = 1 + z + \frac{z^2}{2} + \frac{z^3}{6} + \frac{z^4}{24}, \quad (20)$$

$$R_{\text{IE}}(z) = \frac{1}{1 - z}. \quad (21)$$

For EE, $|1 + z| \leq 1$ is the closed disk of radius 1 centred at -1 ; on the negative real axis this becomes

$$h |\lambda| \leq 2. \quad (22)$$

For RK4 the region extends along the negative real axis to $z = -2.785293$, so for real negative eigenvalues

$$h |\lambda| \leq 2.785293. \quad (23)$$

For IE, $|R_{\text{IE}}(z)| \leq 1$ holds on the whole closed left half-plane: the method is A-stable, and no step-size restriction arises from stability for any λ with $\text{Re } \lambda < 0$.

Applied to this problem: linearising (2) along the computed solution gives the Jacobian (4) with eigenvalues $\lambda_j(t)$, and the *frozen-Jacobian assumption* treats J as constant over a few steps and applies (18) with the measured $\lambda_j(t)$. We are explicit about what this is: a local linear estimate, not a nonlinear stability theorem; J varies along the trajectory, and for non-normal Jacobians eigenvalues alone can mislead. The eigenvalues are real at all sampled output times (Section 1.2); the narrow complex interval noted there does not affect the bounds below, but Section 4.2 still verifies every prediction against an empirical step sweep.

The binding mode is the fast one. With $|\lambda|_{\max}(40) = 3392.79$ (Figure 3), the bounds (22) and (23) evaluated at the end of the integration read

$$h_{\text{EE}} < \frac{2}{3392.79} = 5.89 \times 10^{-4}, \quad h_{\text{RK4}} < \frac{2.785293}{3392.79} = 8.21 \times 10^{-4}. \quad (24)$$

IE has no such bound. The structural zero eigenvalue of (4) is harmless: $R(0) = 1$ for all three methods, so the neutral mode is neither damped nor amplified by design. Figure 4 shows the three stability regions with the curve $h\lambda_{\max}(t)$ overlaid for representative step sizes: for EE with $h = 5 \times 10^{-4}$ the curve stays inside for the whole integration, while $h = 6.5 \times 10^{-4}$ exits through the negative real axis; for RK4 the pair is 8×10^{-4} (inside) versus 9.5×10^{-4} (outside); for IE the curve never leaves the region at any h .

2.4 Convergence

For one-step methods the connection between the two properties above is the classical convergence theorem for one-step methods (see [5, §II.3] and [6]): if Φ_h has local error $O(h^{p+1})$ and satisfies the one-step Lipschitz condition $\|\Phi_h(u) - \Phi_h(v)\| \leq (1 + Lh) \|u - v\|$, then the global error satisfies

$$\|y_N - y(T)\| \leq C h^p, \quad Nh = T, \quad (25)$$

with C depending on T , on the Lipschitz data of f , and on the method, but not on h . The hypotheses check out concretely: the vector field (2) is polynomial, hence locally Lipschitz; on the invariant simplex $\{y_i \geq 0, \sum_i y_i = 1\}$ —which the exact solution never leaves— f and J are bounded, so a global Lipschitz constant exists there. EE and RK4 inherit the Lipschitz constant directly; for IE the Newton matrix in (12) has eigenvalues $1 - h\lambda_j \neq 0$ whenever $\text{Re } \lambda_j \leq 0$, so $I - hJ$ is nonsingular with bounded inverse on the simplex. With consistency from (14)–(16), the theorem predicts global orders 1, 1, 4 for EE, IE, RK4, and Section 4 confirms 1.00, 1.0, 4 wherever runs are stable and the reference noise floor is not reached.

The caveat that makes this theorem almost vacuous for stiff problems is the size of C : it grows with $\|J\|$, so “convergence at order p ” becomes visible for the explicit methods only once h satisfies (24)—on $[0, 40]$ that means 6.8×10^4 steps at the stability boundary for EE, and far more for a first-order-accurate answer. This gap between asymptotic convergence and practical stability is the entire reason implicit methods exist, and Section 4.4 quantifies it in wall-clock terms.

3. IMPLEMENTATION

Section 2 says what the methods should do; this section describes how we made them do it. All code is pure Python with NumPy [7]; SciPy [13] is used only as an independent oracle, never inside our integrators.

3.1 Algorithm

Algorithm 1 is the implicit-Euler stepper with Newton iteration; Algorithm 2 is the adaptive step-doubling controller built on top of it.

Algorithm 1 Implicit Euler with Newton iteration (fixed step)

Require: right-hand side $f(t, y)$, Jacobian $J(t, y)$, initial data (t_0, y_0) , step h , final time T , Newton tolerance τ_N , max iterations K

Ensure: approximate solution y_N at $t_N = T$

```

1:  $t \leftarrow t_0$ ;  $y \leftarrow y_0$ 
2: while  $t < T$  do
3:    $h \leftarrow \min(h, T - t)$ 
4:    $z \leftarrow y + h f(t, y)$  ▷ initial guess: one explicit-Euler predictor
5:   for  $k = 0, 1, \dots, K - 1$  do
6:      $G \leftarrow z - y - h f(t + h, z)$ 
7:     if  $\|G\|_\infty \leq \tau_N$  then exit loop
8:   end if
9:   solve  $(I - hJ(t + h, z)) \Delta z = -G$ ;  $z \leftarrow z + \Delta z$ 
10:  end for
11:  if loop used  $K$  iterations without converging then
12:     $h \leftarrow h/2$ ; go to step 3 ▷ reject and retry with half step
13:  end if
14:   $t \leftarrow t + h$ ;  $y \leftarrow z$ 
15: end while
16: return  $y$ 

```

The predictor in step 4 costs one evaluation of f at (t_n, y_n) and places the iterate inside Newton's local basin for all step sizes we use, so the loop converges in 1.2–2.2 iterations on average (Section 4.4). The rejection path is exercised only when τ_N is set too loose; at our production tolerance $\tau_N = 10^{-10}$ it never triggers on this problem.

Algorithm 2 Adaptive step-doubling controller for implicit Euler

Require: $f, J, (t_0, y_0), T$, rtol, initial step h_0 , safety 0.9, factors $[0.3, 2]$

Ensure: solution values at accepted times, final error statistics

```

1:  $t \leftarrow t_0$ ;  $y \leftarrow y_0$ ;  $h \leftarrow h_0$ 
2: while  $t < T$  do
3:    $h \leftarrow \min(h, T - t)$  ▷ never step past the endpoint
4:    $y^{(1)} \leftarrow \text{IE\_STEP}(t, y, h)$  ▷ one full step of size  $h$ 
5:    $y^{(m)} \leftarrow \text{IE\_STEP}(t, y, h/2)$ ;  $y^{(2)} \leftarrow \text{IE\_STEP}(t + h/2, y^{(m)}, h/2)$  ▷ two half steps
6:   if any of the three IE_STEP calls returned failure then
7:      $h \leftarrow h/2$ ; continue ▷ reject the whole trial; recompute from the original  $(t, y)$ 
8:   end if
9:    $\text{est} \leftarrow \max_i |y_i^{(2)} - y_i^{(1)}| / (\text{atol} + \text{rtol} |y_i^{(2)}|)$  ▷ normalised estimate
10:  if  $\text{est} \leq 1$  then
11:     $t \leftarrow t + h$ ;  $y \leftarrow y^{(2)}$  ▷ accept the more accurate value
12:  end if
13:   $h \leftarrow \text{clip}(0.9 h (\max(\text{est}, \text{tiny}))^{-1/2}, 0.3h, 2h)$ 
14: end while
15: return accepted  $(t, y)$  history

```

Step doubling estimates the local error by comparing one step of size h against two of size $h/2$. For a method of order p the local errors differ by a factor 2^p , so the difference estimates the error of the finer computation:

$$\text{est} = \|y^{(2)} - y^{(1)}\|_w \approx (2^p - 1) \|e_{h/2}\|_w, \quad (26)$$

which for IE ($p = 1$) means the difference directly approximates the local error of the two-half-step result. The norm $\|v\|_w = \max_i |v_i| / (\text{atol} + \text{rtol} |y_i^{(2)}|)$ weights the components so that the small intermediate $y_2 \sim 10^{-5}$ is not swamped by $y_1 \sim 1$; we use $\text{atol} = 10^{-8}$ in all adaptive experiments, and with this normalisation the acceptance test is simply $\text{est} \leq 1$. The controller is an I-controller: since the next local error scales like h^{p+1} , the step is updated by

$$h_{\text{new}} = 0.9 h \text{est}^{-1/(p+1)}, \quad p = 1, \quad (27)$$

clipped to $[0.3, 2]$ to avoid oscillatory step sequences. Two error measures should not be confused: this per-step *local* estimate est (dimensionless, tolerance-relative) and the *global* endpoint metric $\max_i |\Delta y_i| / (10^{-8} + 10^{-6} |y_i^{\text{ref}}|)$

of Section 4.4, used for all method comparisons. The safety factor 0.9 pays for the crudeness of the estimate; the exponent $1/2$ is the order-one signature. A rejected step reuses the same formula with the already-computed estimate, so a rejection costs the three solves but no extra logic. Inside this controller $\text{IE_STEP}(t, y, h)$ has a strict contract: it either completes the full step of the requested size h , or it reports failure—it never advances the state by a silently halved step. If any of the three calls fails (Newton exhausts its iteration cap), the controller rejects the whole trial, halves h , and recomputes all three solves from the original (t, y) , so the accepted state and the time axis cannot drift apart. The clamp $h \leftarrow \min(h, T - t)$ at the top of the loop makes the final step land exactly on $T = 40$ instead of overshooting it.

3.2 Code Structure

Table 3 organises the codebase; running `run_all.py` regenerates every number, table, and figure in this report in a few minutes, and all 10 unit tests pass (Section 4.1).

Table 3: Organisation of the codebase.

File	Purpose
<code>methods.py</code>	core time-stepping: EE, RK4, IE with Newton (Algorithm 1)
<code>model.py</code>	the right-hand side $f(t, y)$ of (2) and the analytic Jacobian (4)
<code>adaptive.py</code>	adaptive step-doubling controller (Algorithm 2) with the I-controller (27)
<code>experiments.py</code>	experiment drivers: PR order test, convergence, threshold and work–precision sweeps
<code>validation_tests.py</code>	PASS/FAIL unit-test suite (10 tests, Section 4.1)
<code>run_all.py</code>	reproduces every number, table, and figure from scratch
<code>make_figures.py</code>	reads <code>results/*.npz</code> and renders the figures of Section 4

3.3 Key Design Decisions

Four choices deserve comment.

Newton initial guess. We use the explicit-Euler predictor $z^{(0)} = y_n + hf(t_n, y_n)$ rather than the previous value y_n . The predictor costs one function evaluation but removes the $O(h)$ discrepancy between y_n and the solution of (11) before Newton starts, which is why the measured count stays at 1.2–2.2 iterations per step.

Newton stopping criterion. We stop on the residual $\|G(z)\|_\infty \leq \tau_N$ with $\tau_N = 10^{-10}$, not on the increment $\|\Delta z\|$. The residual is the quantity that enters the invariant defect and the local error, so it is the honest thing to bound. The sweep of Section 4.4 justifies the choice: at $\tau_N = 10^{-6}$ the global error is unchanged, at $\tau_N = 10^{-4}$ the run is wrong by four orders of magnitude while still conserving mass to 3.8×10^{-15} , and at $\tau_N = 10^{-2}$ the iteration diverges entirely.

What we implemented versus what we delegated. All integrators, the Newton solver, the step-doubling estimator, and the controller are our own code; the 3×3 Newton systems use `numpy.linalg.solve`. SciPy appears in exactly one place: `solve_ivp` with Radau and BDF [6, 13] supplies the reference solution and the oracle rows of the work–precision table. The split is methodological: an independent, professionally tested solver is the right oracle for verification, but letting it do the integration would remove the object of study.

Vectorisation. The Robertson state has three components, so there is nothing to batch; our loops are plain Python with NumPy for the 3×3 algebra. The cost shows in Section 4.4’s wall times; it affects no order, stability, or adaptivity conclusion (Section 4.5, item 2).

4. NUMERICAL EXPERIMENTS

We test the methods on the Robertson problem over $0 \leq t \leq 40$. The experiments assess convergence, stability, adaptivity and computational cost. Figure 1 shows the reference trajectory.

4.1 Reference Solution and Verification

Reference solution. We used SciPy Radau [6, 13] with $\text{rtol} = 10^{-12}$ and $\text{atol} = 10^{-14}$. The common output grid contains $t = 0$ and 241 logarithmically spaced times from 10^{-8} to 40. Internal solver steps are independent of these

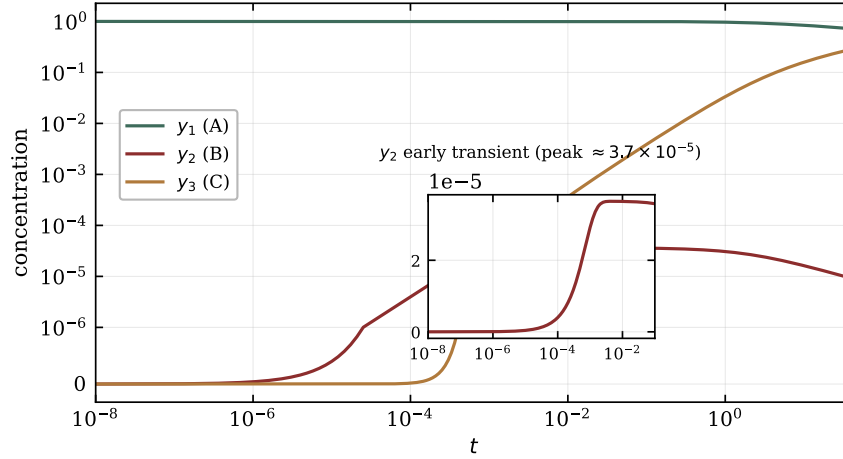


Figure 1: Reference solution on a logarithmic time axis. The intermediate concentration y_2 peaks at approximately 3.65×10^{-5} near $t = 0.0046$ and then decreases. The solution changes more slowly after the initial transient, although the fast mode still restricts explicit step sizes.

output times.

We verified the reference solution by repeating the Radau calculation with $(\text{rtol}, \text{atol}) = (10^{-10}, 10^{-12})$ and $(10^{-12}, 10^{-14})$. Both tolerances were reduced by exactly a factor of 100, with no adjustment by the solver. The two solutions were evaluated at $t = 0$ and 241 common logarithmically spaced times from 10^{-8} to 40, while Radau selected its own internal steps. The tighter solution was retained as the reference. The maximum absolute difference over all components and output times was 4.950×10^{-12} . At $t = 40$, the componentwise differences were 2.738×10^{-12} , 1.031×10^{-16} , and 2.739×10^{-12} , respectively. Both Radau runs and an independent BDF calculation agreed after rounding each endpoint component to 10 significant digits, supporting the values reported in (28). These comparisons establish numerical consistency, rather than a rigorous error bound.

$$y(40) = (0.7158270687, \quad 9.185534765 \times 10^{-6}, \quad 0.2841637457). \quad (28)$$

This result agrees with the approximate check values in the project pack.

Table 4: Reference check under a 100-fold tolerance tightening: R0 = Radau at $(\text{rtol}, \text{atol}) = (10^{-10}, 10^{-12})$ versus R1 = Radau at $(10^{-12}, 10^{-14})$, the latter retained as the reference. *Matching rounded digits* are the leading significant digits that remain identical after the R0 and R1 values are actually rounded—a statement of numerical agreement, not a rigorous true-error bound. The displayed reference values keep the 10 significant digits confirmed by both Radau runs and the independent BDF check.

component	reference at $t = 40$ (displayed values)	absolute change		relative change	matching rounded digits (R0 / R1)
		at $t = 40$	max on output grid	at $t = 40$	
y_1	0.7158270687	2.738×10^{-12}	4.948×10^{-12}	3.825×10^{-12}	11
y_2	$9.185534765 \times 10^{-6}$	1.031×10^{-16}	2.180×10^{-13}	1.123×10^{-11}	10
y_3	0.2841637457	2.739×10^{-12}	4.950×10^{-12}	9.640×10^{-12}	10

Order verification. We used the Prothero–Robinson test [9], $\dot{y} = \lambda(y - e^{-t}) - e^{-t}$, with $\lambda = -1$ and $T = 1$. Here the equation reduces to $\dot{y} = -y$, with exact solution e^{-t} . Table 5 confirms orders approaching 1, 1 and 4 for EE, IE and RK4. In the stiff variant with $\lambda = -10^4$, EE and RK4 become unstable at the tested steps, while IE gives an observed order of about 1.03.

Implementation checks. The production validation suite contains ten passing tests; a separate eight-test suite on the independent validation branch (code/validation/) re-runs the core checks against re-implemented solvers. Checks include the conservation identity, the analytic Jacobian, Newton convergence, scalar convergence orders and tolerance refinement. The Jacobian agrees with a complex-step derivative to 9.1×10^{-13} . The IE solution at $h = 10^{-4}$ differs from Radau by 4.9×10^{-7} at $t = 1$. These tests check the implementation independently of the Robertson endpoint comparison.

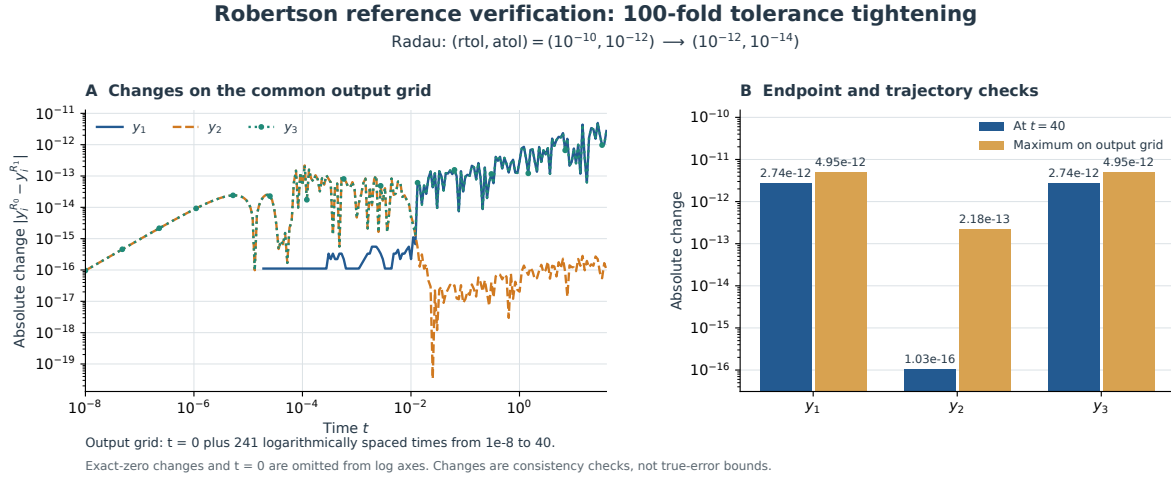


Figure 2: Verification of the reference solution under the 100-fold tolerance tightening (rtol, atol) = $(10^{-10}, 10^{-12}) \rightarrow (10^{-12}, 10^{-14})$. Panel A: absolute change of each component on the common output grid ($t = 0$ plus 241 logarithmically spaced times from 10^{-8} to 40; exact-zero changes and $t = 0$ are omitted from the logarithmic axes). Panel B: componentwise changes at $t = 40$ and maxima over the grid. All changes lie at or below 4.950×10^{-12} , consistent with the 10 rounded digits quoted in (28); the comparison is a consistency check, not a true-error bound.

Table 5: Errors and observed orders for the scalar test with $\lambda = -1$ and $T = 1$. Orders are calculated from successive error ratios.

h	EE error	EE order	IE error	IE order	RK4 error	RK4 order
0.2	4.020×10^{-2}	—	3.400×10^{-2}	—	5.797×10^{-6}	—
0.1	1.920×10^{-2}	1.07	1.766×10^{-2}	0.94	3.332×10^{-7}	4.12
0.05	9.394×10^{-3}	1.03	9.010×10^{-3}	0.97	1.998×10^{-8}	4.06
0.025	4.647×10^{-3}	1.02	4.551×10^{-3}	0.99	1.223×10^{-9}	4.03
0.0125	2.311×10^{-3}	1.01	2.287×10^{-3}	0.99	7.563×10^{-11}	4.02

4.2 Stability Experiments

We compare the local frozen-Jacobian estimates in (24) with an empirical step-size sweep. Figure 3 shows the eigenvalue magnitudes and the stiffness ratio used in this comparison.

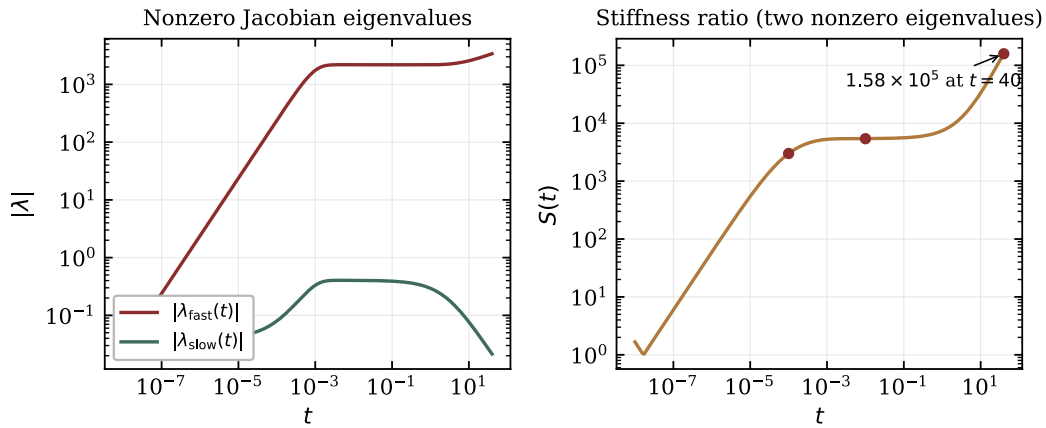


Figure 3: Magnitudes of the two nonzero Jacobian eigenvalues and the stiffness ratio (5). The fast magnitude increases from 239.47 at $t = 10^{-4}$ to 3392.79 at $t = 40$; the corresponding stiffness ratios are 2.996×10^3 and 1.5840×10^5 . The late-time fast mode gives the most restrictive explicit step-size estimate.

For EE, the weighted endpoint error is 7.07 at $h = 5.95 \times 10^{-4}$, but rises to 1.08×10^4 at 6.0×10^{-4} . The latter run reaches a negative concentration of -4.84×10^{-6} at $t = 38.95$. Steps of 6.3×10^{-4} and above produce non-finite values in the reported sweep. The observed loss of accuracy is within 1–2% of the local estimate, 5.89×10^{-4} .

Crossing the local boundary does not imply immediate overflow. At $h = 6.5 \times 10^{-4}$, the reference eigenvalue

crosses $2/h$ at $t = 30.34$, whereas overflow occurs at $t = 34.64$. This delay shows why the frozen-Jacobian estimate must be checked against the numerical trajectory.

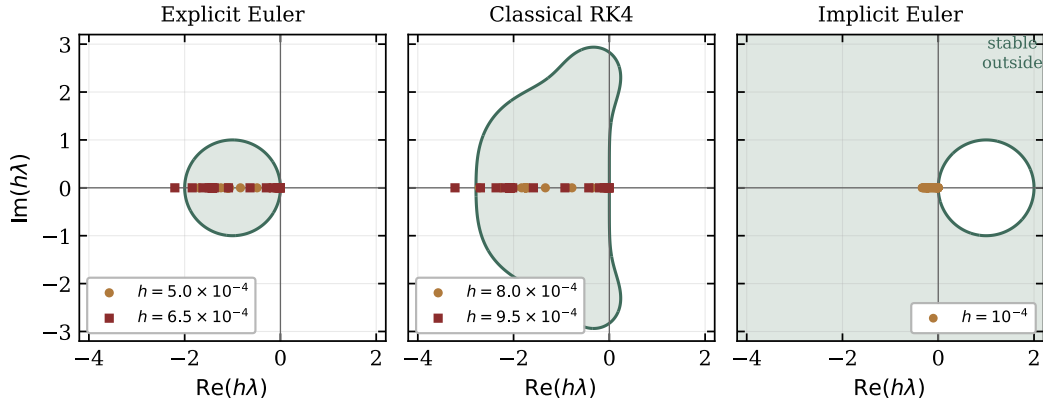


Figure 4: Absolute-stability regions with representative $h\lambda$ trajectories. For EE, $h = 5 \times 10^{-4}$ stays inside the region, but its error exceeds the accuracy target in Table 7. At 6.5×10^{-4} , the trajectory leaves the region and the run diverges. RK4 shows the same distinction at 8×10^{-4} and 9.5×10^{-4} . IE contains the entire left half-plane. These overlays are local linear diagnostics.

RK4 gives an error of approximately 3.6×10^{-5} at $h = 8.2 \times 10^{-4}$, an error of 9176 at 8.5×10^{-4} , and non-finite values at 10^{-3} . Its predicted boundary is 8.21×10^{-4} . IE remains bounded at $h = 0.25$, about 425 times the EE limit, but its large error in Table 7 shows that stability alone does not establish accuracy.

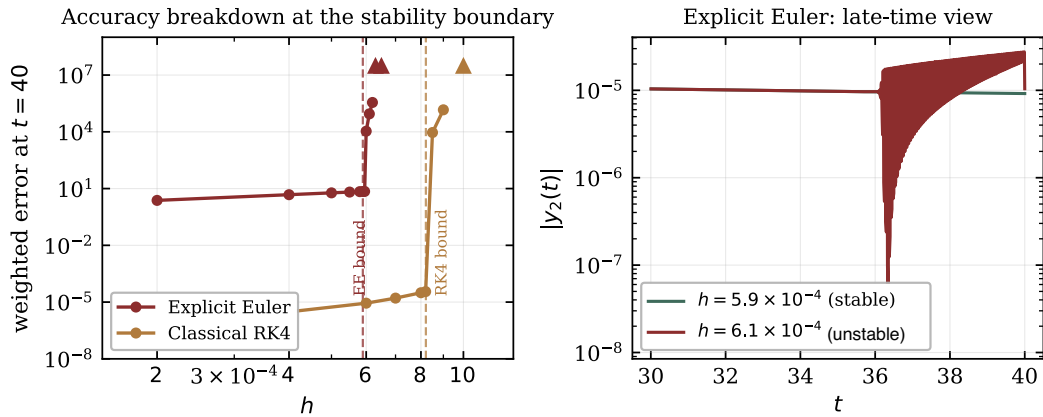


Figure 5: Left: endpoint error against step size, with the local estimates (24). Right: late-time $|y_2|$ for two EE runs. The red run at $h = 6.1 \times 10^{-4}$ develops numerical oscillations and negative values; the absolute-value plot hides the sign changes. Dense connections between successive samples produce the shaded appearance. Stability, accuracy and non-negativity are assessed separately.

4.3 Adaptivity

The adaptive IE solver was tested at four tolerances (Table 6). Each run rejects only three or four trial steps. Reducing rtol by a factor of ten decreases the endpoint error by approximately a factor of three, close to $\sqrt{10}$. This is consistent with first-order global accuracy and a second-order local error estimate when the relative-tolerance term controls the step size. It does not imply that the global error equals the requested local tolerance.

4.4 Accuracy and Computational Cost

We examine convergence and the final intermediate concentration, then compare computational cost. All methods use the same weighted endpoint error:

$$E = \max_{i=1,2,3} \frac{|y_i(40) - y_i^{\text{ref}}(40)|}{10^{-8} + 10^{-6} |y_i^{\text{ref}}(40)|}.$$

Table 6: Adaptive IE with step doubling, $T = 40$ and $\text{atol} = 10^{-8}$. The error column uses the fixed comparison norm defined in Section 4.4. Wall times are single-run measurements; Figure 9 reports minima over repeated runs, hence the small difference from the 0.648 s of Section 4.4.

rtol	accepted	rejected	f -evals	time [s]	$h_{\min}$	$h_{\max}$	error	$y_2(40)$
10^{-3}	136	3	1358	0.056	10^{-6}	3.49	5.95×10^3	9.2539×10^{-6}
10^{-4}	315	3	2918	0.113	10^{-6}	1.26	2.01×10^3	9.2086×10^{-6}
10^{-5}	746	3	6727	0.249	10^{-6}	0.405	6.42×10^2	9.1929×10^{-6}
10^{-6}	1934	4	17428	0.662	10^{-6}	0.130	2.05×10^2	9.1879×10^{-6}

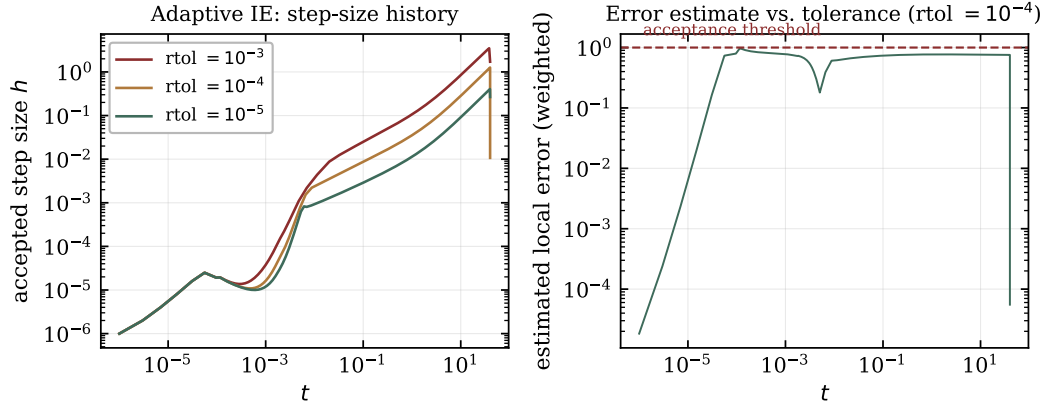


Figure 6: Left: accepted step sizes for three tolerances. Small steps resolve the initial transient; larger steps follow the slower evolution. Right: the normalised local error estimate for $\text{rtol} = 10^{-4}$ remains below the acceptance threshold of one. This plot checks local error control, not the global endpoint error.

The comparison weights are fixed; they are distinct from solver tolerances.

Convergence. Halving the step size approximately halves the EE and IE errors (Table 7). The RK4 errors instead bottom out near 10^{-6} and rise again for the two finest grids (2.41×10^{-6} and 6.15×10^{-6}). This floor is not reference uncertainty — under a 100-fold tolerance tightening the reference moved by at most 4.950×10^{-12} (Table 4) — but accumulated round-off amplified by the weighted norm: per-step rounding at the level of unit roundoff accumulates over the 6.4×10^5 steps of the finest run to an absolute contamination of order 10^{-14} in $y_2(40)$, which the small weight $10^{-8} + 10^{-6} |y_2^{\text{ref}}| \approx 10^{-8}$ inflates to a weighted error of order 10^{-6} . The floor therefore grows with the step count, which is why the two finest RK4 runs move upward. On the shorter interval $[0, 0.01]$, where far fewer steps accumulate, the same data give refinement ratios 264.8, 20.6 and 17.5; the last two approach the fourth-order ratio of 16, so above this noise floor RK4 does converge at its design order.

Table 7: Uniform-step convergence at $t = 40$. Mass defect is the maximum of $|y_1 + y_2 + y_3 - 1|$ along the run. Each method uses at least four successively halved step sizes.

Method	h	steps N	error	stable	non-neg.	accurate	mass defect
IE	0.25	160	2938.9	Yes	Yes	No	2.2×10^{-16}
IE	0.125	320	1476.8	Yes	Yes	No	8.9×10^{-16}
IE	0.0625	640	740.3	Yes	Yes	No	8.9×10^{-16}
IE	0.03125	1280	370.6	Yes	Yes	No	1.1×10^{-15}
IE	0.015625	2560	185.4	Yes	Yes	No	3.8×10^{-15}
IE	7.8125×10^{-3}	5120	92.7	Yes	Yes	No	2.9×10^{-15}
EE	5×10^{-4}	8×10^4	5.938	Yes	Yes	No	1.3×10^{-14}
EE	2.5×10^{-4}	1.6×10^5	2.969	Yes	Yes	No	1.0×10^{-14}
EE	1.25×10^{-4}	3.2×10^5	1.485	Yes	Yes	No	1.2×10^{-14}
EE	6.25×10^{-5}	6.4×10^5	0.7423	Yes	Yes	Yes	3.7×10^{-14}
RK4	5×10^{-4}	8×10^4	4.41×10^{-6}	Yes	Yes	Yes	1.1×10^{-14}
RK4	2.5×10^{-4}	1.6×10^5	9.88×10^{-7}	Yes	Yes	Yes	1.2×10^{-14}
RK4	1.25×10^{-4}	3.2×10^5	2.41×10^{-6}	Yes	Yes	Yes	1.2×10^{-14}
RK4	6.25×10^{-5}	6.4×10^5	6.15×10^{-6}	Yes	Yes	Yes	2.8×10^{-14}

For Table 7, “stable” means that all recorded states are finite and $\max_i |y_i| \leq 10$. This is an empirical bound-ness check, not a nonlinear stability theorem. “Non-negative” requires every component to remain at least zero, and “accurate” requires endpoint error ≤ 1 . All listed runs pass the first two checks; mass is conserved to within 3.7×10^{-14} by the EE and RK4 runs and to within 3.8×10^{-15} by the IE runs. Only the finest EE run and the RK4 runs pass the accuracy check; none of the coarse IE runs do.

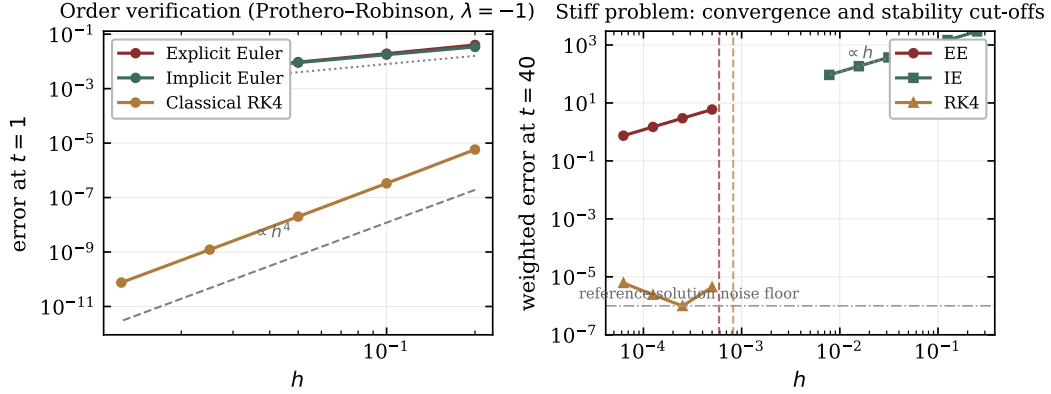


Figure 7: Left: scalar-test errors approach orders 1, 1 and 4. Right: Robertson endpoint errors for Table 7. Vertical lines mark the local explicit stability estimates; the horizontal line indicates the approximate reference-error scale. EE and IE show first-order trends. The RK4 data at this endpoint do not resolve a fourth-order trend; the weighted-norm round-off floor responsible is analysed above.

Final intermediate concentration. The value of $y_2(40)$ is about five orders of magnitude smaller than $y_1(40)$, so we check it separately. Table 8 shows IE approaching the reference as the step size or tolerance decreases. Radau runs at different tolerances provide a consistency check; BDF supplies the independent method comparison.

Table 8: Cross-check of $y_2(40)$ against (28). Digits denote $-\log_{10}$ of the relative error. The reference run itself is omitted.

Method / setting	$y_2(40)$	digits
IE adaptive, $\text{rtol} = 10^{-3} / 10^{-4} / 10^{-5} / 10^{-6}$	$(9.2539 / 9.2086 / 9.1929 / 9.1879) \times 10^{-6}$	2.1 / 2.6 / 3.1 / 3.6
IE uniform, $N = 160 / 640 / 2560$	$(9.2192 / 9.1940 / 9.1877) \times 10^{-6}$	2.4 / 3.0 / 3.6
Radau, $\text{rtol} = 10^{-6} / 10^{-9}$	$(9.1855347624 / 9.18553476456) \times 10^{-6}$	9.6 / 11.6
BDF, $\text{rtol} = 10^{-12}$	$9.1855347651 \times 10^{-6}$	10.2

Conservation and Newton tolerance. The mass defect stays below 4×10^{-14} for all three methods at $h = 10^{-4}$ and for IE at $h = 0.05$. The explicit methods preserve the linear invariant algebraically. With the stated predictor and exact linear solves, each Newton correction also preserves it because $(1, 1, 1)^T J = 0$. Small mass defect therefore does not prove that Newton has converged.

At $h = 0.05$ and $\tau_N = 10^{-4}$, IE conserves mass to 3.77×10^{-15} but has endpoint error 8.47×10^6 and $y_2(40) = -2.5663 \times 10^{-6}$; at $\tau_N = 10^{-2}$ the iteration diverges. Tightening τ_N from 10^{-6} to 10^{-13} leaves the error at approximately 592.6 while the mean iteration count grows from 1.23 to 2.22 (full sweep in Appendix C.1), supporting $\tau_N = 10^{-10}$ for the production runs at this step size; finer-step benchmarks require their own check.

Work-precision comparison. Figure 9 plots weighted endpoint error against the minimum wall time from repeated runs in Python 3.12. In the tested configurations, Radau gives a better cost-accuracy trade-off than the implemented EE, RK4 and IE methods. For example, Radau reaches error 1.75×10^{-4} in 0.040 s, whereas adaptive IE reaches error 205 in about 0.65 s. These different-error results show the overall trend, not a matched-accuracy speed-up factor.

Uniform IE reaches error 92.7 in 0.554 s, compared with 205 in 0.648 s for adaptive IE. Step doubling requires three implicit solves per trial, so larger later steps do not necessarily reduce runtime. In this series, EE reaches error 0.742 in 4.74 s, while RK4 takes 2.2–18.2 s for errors near the round-off floor.

Matched endpoint accuracy. A separate supplementary benchmark compares EE and IE at measured errors of approximately 0.7423. Both use 640,000 steps ($h = 6.25 \times 10^{-5}$). EE takes 0.98 s and IE takes 5.07 s. Across the tested targets, the IE/EE runtime ratio is 5.16–5.25. At these endpoint accuracies, IE does not gain a runtime advantage from its linear stability; the ratio measures algorithm and implementation together.

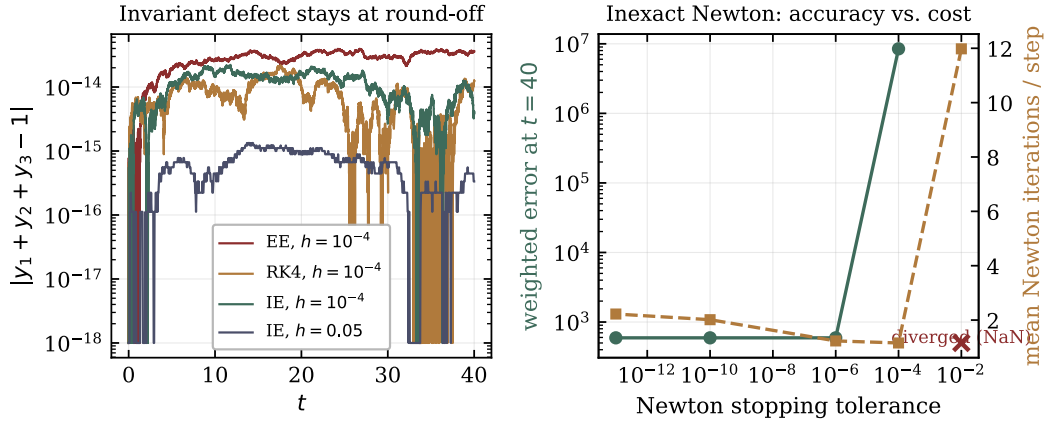


Figure 8: Left: mass-conservation defects remain near round-off. Right: at $h = 0.05$, tightening the Newton tolerance eventually stops reducing endpoint error, while the iteration count increases. The plateau indicates that time-discretisation error dominates. A loose nonlinear tolerance can give negative and inaccurate solutions despite small mass defect.

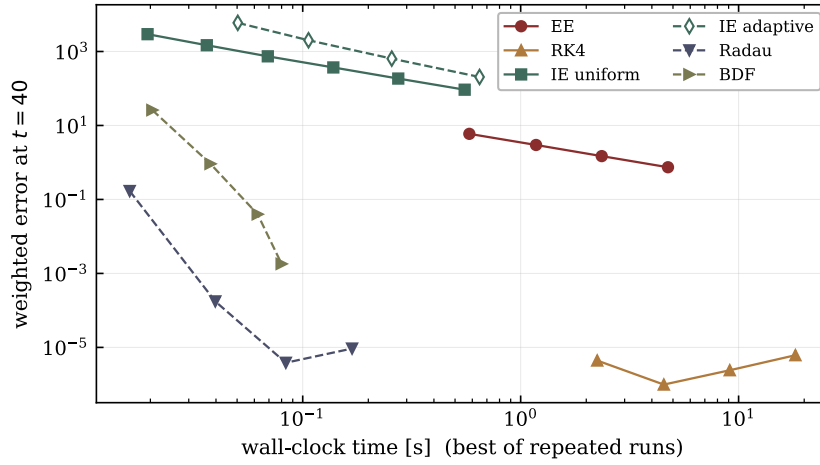


Figure 9: Weighted endpoint error versus wall time for EE, RK4, uniform and adaptive IE, and SciPy Radau and BDF. Lower errors and shorter runtimes indicate better performance. The curves show the tested configurations; a fixed speed-up factor requires comparison at matched errors. Wall times are minima over repeated runs, measured with Python 3.12.14, NumPy 2.2.6 and SciPy 1.15.3 on macOS 26.6.2 (arm64).

The protocol is recorded in Appendix C.2: the measured EE errors at 80,000, 160,000, 320,000 and 640,000 steps defined the four targets E^* , an IE run was accepted as matched when $0.95E^* \leq E_{IE} \leq E^*$, and each configuration was warmed up once and timed five times with the minimum reported. IE matched each target at the same step count as EE, so the measured ratio is essentially the per-step Newton cost. These timings belong to this benchmark and should not be combined with the Figure 9 series.

Coarse accuracy targets. The four targets span only the relatively accurate band $0.74 \leq E^* \leq 5.94$. At the coarse end the comparison is one-sided: EE cannot trade accuracy for speed, because stability pins every usable run to $h \lesssim 5.9 \times 10^{-4}$ (about 6.7×10^4 steps on $[0, 40]$; larger steps are inaccurate or non-finite, Section 4.2), whereas IE meets the error 592.6 with 800 steps at $h = 0.05$ and stays stable at $h = 0.25$ with 160 steps (Table 7) — 84–420 times fewer steps, so even with the per-step Newton cost IE is over an order of magnitude cheaper at loose targets. The 5.16–5.25 ratio is therefore the high-accuracy statement: each method has an accuracy regime in which it wins.

4.5 Limitations

1. First-order accuracy. IE needs small steps to achieve a small global error even when stability permits larger ones. A higher-order stiff method, such as BDF2 or a suitable Rosenbrock method [11], is a possible improvement; its benefit would need to be measured.
2. Timing scope. Runtime depends on the implementation, hardware and benchmark protocol. The results do not isolate Python overhead from algorithmic cost, and neither absolute times nor runtime ratios are universal.

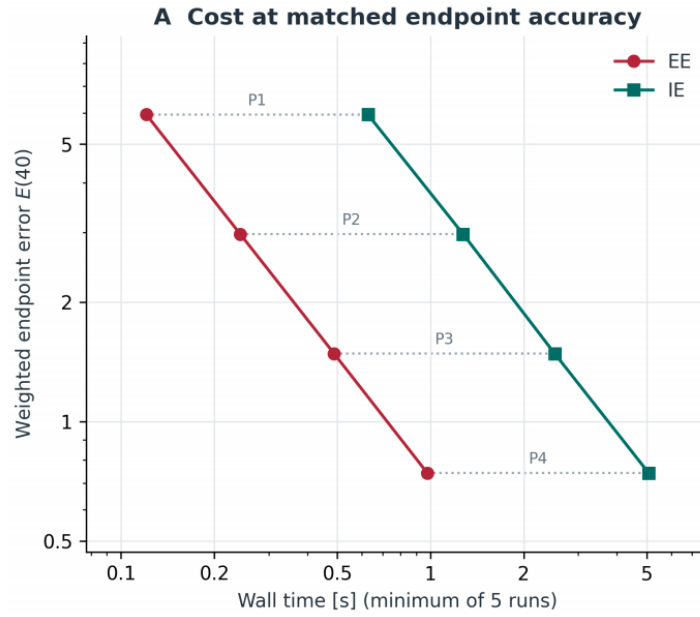


Figure 10: EE and IE runtime at matched weighted endpoint errors. Dotted lines connect pairs whose errors agree within 5%. Runtime is the minimum of five runs after warm-up. IE required 5.16–5.25 times the EE runtime across the four tested targets.

3. Local stability estimates. The predicted explicit thresholds agree with the reported sweep to within 1–2%, but the frozen-Jacobian analysis is not a nonlinear stability proof. A real spectrum also does not rule out non-normal effects.
4. Separate diagnostics. Boundedness, non-negativity, conservation and accuracy answer different questions. The loose-Newton experiment shows that mass conservation alone is insufficient.
5. Reference uncertainty. All reported errors are measured against a numerical reference. The 100-fold Radau tolerance refinement and the independent BDF agreement provide consistency checks, not rigorous error bounds. Errors near the reference-error scale cannot establish further convergence; the RK4 floor in Table 7 is instead round-off amplified by the small weight on y_2 (Section 4.4).

5. CONCLUSIONS

We implemented EE, RK4 and IE with Newton iteration and applied them to the Robertson problem on $[0, 40]$. A scalar test confirms their expected orders of 1, 4 and 1. Reference refinement, an independent BDF calculation and conservation checks support the numerical comparisons.

The experiments separate stability from accuracy. EE and RK4 require small steps to avoid instability, and the local Jacobian estimates agree closely with the observed thresholds. IE remains bounded at much larger steps, but those steps give large errors. The cost ranking depends on the accuracy regime: at coarse targets IE is substantially cheaper — stability pins EE to $h \lesssim 5.9 \times 10^{-4}$ whatever the target, while IE can use steps up to $h = 0.25$ — and at the strict matched-error targets of the supplementary benchmark the per-step Newton cost reverses the ranking, with EE faster than IE by a factor of 5.16–5.25 for the tested implementation.

Adaptive IE resolves the early transient with small steps and increases the step size later. Tightening the tolerance reduces the endpoint error over the tested range, but the additional step-doubling solves limit runtime savings. The Newton-tolerance sweep also shows that conservation can remain accurate when the numerical solution is negative or otherwise incorrect.

Future work should test a higher-order stiff method, improve step-size control and assess positivity-preserving rejection. Further performance claims should use a documented timing protocol and matched accuracy targets.

REFERENCES

- [1] P. N. Brown, G. D. Byrne, and A. C. Hindmarsh, “VODE: A variable coefficient ODE solver,” *SIAM J. Sci. Stat. Comput.*, vol. 10, no. 5, pp. 1038–1051, 1989, doi: [10.1137/0910062](https://doi.org/10.1137/0910062).
- [2] G. D. Byrne and A. C. Hindmarsh, “Stiff ODE solvers: A review of current and coming attractions,” *J. Comput. Phys.*, vol. 70, no. 1, pp. 1–62, 1987, doi: [10.1016/0021-9991\(87\)90001-5](https://doi.org/10.1016/0021-9991(87)90001-5).
- [3] C. F. Curtiss and J. O. Hirschfelder, “Integration of stiff equations,” *Proc. Natl. Acad. Sci. USA*, vol. 38, no. 3, pp. 235–243, 1952, doi: [10.1073/pnas.38.3.235](https://doi.org/10.1073/pnas.38.3.235).
- [4] C. W. Gear, *Numerical Initial Value Problems in Ordinary Differential Equations*. Prentice-Hall, 1971.
- [5] E. Hairer, S. P. Nørsett, and G. Wanner, *Solving Ordinary Differential Equations I: Nonstiff Problems*, 2nd ed. Springer, 1993, doi: [10.1007/978-3-540-78862-1](https://doi.org/10.1007/978-3-540-78862-1).
- [6] E. Hairer and G. Wanner, *Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems*, 2nd ed. Springer, 1996, doi: [10.1007/978-3-642-05221-7](https://doi.org/10.1007/978-3-642-05221-7).
- [7] C. R. Harris et al., “Array programming with NumPy,” *Nature*, vol. 585, no. 7825, pp. 357–362, 2020, doi: [10.1038/s41586-020-2649-2](https://doi.org/10.1038/s41586-020-2649-2).
- [8] A. C. Hindmarsh et al., “SUNDIALS: Suite of nonlinear and differential/algebraic equation solvers,” *ACM Trans. Math. Softw.*, vol. 31, no. 3, pp. 363–396, 2005, doi: [10.1145/1089014.1089020](https://doi.org/10.1145/1089014.1089020).
- [9] A. Prothero and A. Robinson, “On the stability and accuracy of one-step methods for solving stiff systems of ordinary differential equations,” *Math. Comp.*, vol. 28, no. 125, pp. 145–162, 1974, doi: [10.1090/S0025-5718-1974-0331793-2](https://doi.org/10.1090/S0025-5718-1974-0331793-2).
- [10] H. H. Robertson, “The solution of a set of reaction rate equations,” in *Numerical Analysis: An Introduction*, J. Walsh, Ed. Academic Press, 1966, pp. 178–182.
- [11] A. Sandu et al., “Benchmarking stiff ODE solvers for atmospheric chemistry problems—II. Rosenbrock solvers,” *Atmos. Environ.*, vol. 31, no. 20, pp. 3459–3472, 1997, doi: [10.1016/S1352-2310\(97\)83212-8](https://doi.org/10.1016/S1352-2310(97)83212-8).
- [12] L. F. Shampine and M. W. Reichelt, “The MATLAB ODE suite,” *SIAM J. Sci. Comput.*, vol. 18, no. 1, pp. 1–22, 1997, doi: [10.1137/S1064827594276424](https://doi.org/10.1137/S1064827594276424).
- [13] P. Virtanen et al., “SciPy 1.0: Fundamental algorithms for scientific computing in Python,” *Nat. Methods*, vol. 17, no. 3, pp. 261–272, 2020, doi: [10.1038/s41592-019-0686-2](https://doi.org/10.1038/s41592-019-0686-2).

DEPARTMENT OF APPLIED MATHEMATICS

Project 1 (ODE IVP) Report — Group E

APPENDIX

AI Transparency Log, Individual Contribution Statements, and Supplementary Data

Yi Zhu (2361666) · Zhengdao Song (2361511) · Fangchi Shi (2359837) · Xiangyu Zhang (2361653) · Rong Yin (2363466)

September 2026

This appendix follows the module’s report template: Appendix A is the team-level AI Transparency Log (rubric dimension E, merged from the ICS & AI Transparency Log form), Appendix B collects the five Individual Contribution Statements (rubric dimension F), and Appendix C gathers supplementary tables and data. Section, table and figure numbers refer to the main report.

APPENDIX A — AI TRANSPARENCY LOG

Purpose. This log records the gap between AI output and our understanding. Every AI-assisted claim that entered the report was independently verified by the team; the three audits below document the interactions that materially affected a mathematical claim, an experiment, or an interpretation. Routine interactions are summarised once at the end.

A1. AI tools used

Tool	Date	Primary use
ChatGPT / Codex	Sep 2026	Cross-checking the report against the project brief; designing and executing the supplementary matched-error EE–IE benchmark and the reference-tolerance refinement; revising Sections 4–5 for clarity and consistency. All numerical outputs re-measured before use.
TeleAgent (AI assistant)	Sep 2026	Scaffolding of the mathematical derivation document: method formulas from Taylor expansions, Newton iteration theory, absolute-stability analysis, Jacobian derivation, stiffness analysis. Every mathematical claim re-derived by hand before use.
Kimi AI assistant	Sep 2026	Scaffolding of the validation-suite code (solvers, oracle chain, test harness, plotting boilerplate); debugging of Newton convergence; drafting of figure layouts. Every numerical claim re-measured by our own code before use.
Chat-based LLM	Sep 2026	Matplotlib figure styling and caption drafting; $\LaTeX$ formatting and table layout; language polishing of the abstract and Sections 1.3, 3.2 and 5. No figure data, numerical result or stability claim accepted without checking against <code>results/*.npz</code> and the report tables.

A2–A3. Significant AI-assisted claim audits, independent verification and decision

Item 1 — Matched-error comparison of explicit Euler (EE) and implicit Euler (IE). *Prompt and claim.* AI was asked to design an experimental comparison of explicit Euler (EE) and implicit Euler (IE) at matched endpoint errors. The question under test was whether the larger stable steps available to IE translate into lower computational cost at comparable accuracy. *Risk and possible falsifier.* Fewer steps or shorter runtime at a *larger* error would not establish greater efficiency; the proposed advantage would be contradicted if IE took longer than EE when both met the same measured error target under a common timing protocol. *Independent check and evidence.* AI-assisted scripts were executed for four matched-error EE–IE pairs. Both methods used the same weighted endpoint-error definition; IE was required to meet 95–100% of each EE error target. After warm-up, each configuration was timed five times and the minimum taken. The measured IE/EE runtime ratios were 5.16–5.25. Evidence: Section 4.4, Figure 10, and the saved benchmark data. *Final decision and limitation.* The comparison does not support an IE runtime advantage at these targets. The report states that EE was faster for the tested implementation and endpoint accuracies. The conclusion is limited to this benchmark: it does not establish equal accuracy throughout the trajectory or a universal ranking of the methods.

Item 2 — Reference-solution verification under tolerance refinement. *Prompt and claim.* AI was asked to check whether the reference solution satisfied the required 100-fold tolerance refinement and to generate the missing verification experiment. The reference digits were treated as values requiring verification, not as ground truth. *Risk*

and possible falsifier. Requested tolerances may be adjusted silently by the solver, and apparent agreement may not justify every displayed digit. Possible falsifiers: an effective tolerance reduction smaller than 100, endpoint digits moving under refinement, or disagreement with an independent solver family. *Independent check and evidence.* Radau was run at $(\text{rtol}, \text{atol}) = (10^{-10}, 10^{-12})$ and $(10^{-12}, 10^{-14})$; the effective tolerances were checked, confirming a factor-100 reduction in both, with no adjustment by the solver. The two solutions were compared at $t = 0$ and 241 common logarithmic output times in $[10^{-8}, 40]$; the maximum componentwise change over the grid was 4.950×10^{-12} . An independent BDF calculation provided a separate method check. Evidence: Section 4.1, Table 4 and Figure 2. *Final decision and limitation.* Accepted with restricted precision. Both Radau runs and BDF agreed when each endpoint component was rounded to 10 significant digits, supporting the values retained in equation (28). The report describes this as numerical consistency, not a rigorous bound on the reference error.

Item 3 — AI-assisted mathematical derivations. *Prompt and claim.* AI-assisted derivations claimed (i) that the Robertson Jacobian always has a structural zero eigenvalue arising from the conservation law $y_1 + y_2 + y_3 = 1$ (the column sums vanish identically), and (ii) that implicit Euler is A-stable, $|1/(1 - z)| \leq 1$ for all $\text{Re}(z) < 0$. *Risk and possible falsifier.* Accepting a plausible algebraic argument without checking it at a non-trivial state. Possible falsifiers: eigenvalues computed at a generic state (e.g. $t = 10^{-4}$) with no numerically zero eigenvalue, or a specific z with large negative real part (e.g. $z = -10^6$) violating $|1/(1 - z)| \leq 1$. *Independent check and evidence.* The Jacobian column sums were verified by hand element by element, confirming $(1, 1, 1)$ as a left null vector, and the eigenvalues were checked numerically at a generic state. For A-stability, with $z = a + bi$, $a < 0$: $|1 - z|^2 = (1 - a)^2 + b^2 > 1$, hence $|1/(1 - z)| < 1$; also tested numerically at $z = -10^6$. Evidence: Sections 1.2 and 2.3 of the report and the derivation document. *Final decision and limitation.* Both claims accepted only after hand re-derivation. The stability statements used in the report are local frozen-Jacobian estimates; they were confirmed experimentally to within 1–2% by the step-size sweeps of Section 4.2 and are not claimed as nonlinear stability proofs.

Routine interactions, summarised once. Autocomplete, spelling and L^AT_EX corrections, figure-layout boilerplate and language polishing were used throughout. None of these changed a mathematical claim, a numerical value, or an interpretation.

A4. Intellectual ownership declaration

Each member affirms that they can explain and modify the team-owned numerical methods, justify the mathematical claims, and state the assumptions and tests at library interfaces without AI assistance during the viva.

Name (ID)	Signature	Team-owned artefact / contribution	Trusted dependency and interface
Yi Zhu (2361666)	Yi Zhu	Repository and folder structure; weekly records and milestones; requirements cross-checks; matched-error benchmark experiment.	git/GitHub; AI-assistant outputs, all re-verified before use.
Zhengdao Song (2361511)	Zhengdao Song	Mathematical derivation document (12 sections): model properties, method derivations, Newton theory, stability analysis, Jacobian and stiffness, diagnostic definitions.	NumPy eigenvalue computation only; no external solver for mathematical claims.
Fangchi Shi (2359837)	Fangchi Shi	Production solver framework: <code>methods.py</code> (EE, RK4, IE with Newton), <code>adaptive.py</code> (step-doubling controller), <code>model.py</code> (Robertson RHS and Jacobian); solver diagnostics.	NumPy linear algebra (<code>numpy.linalg.solve</code>) inside Newton.
Xiangyu Zhang (2361653)	Xiangyu Zhang	<code>make_figures.py</code> and Figures 1–10; L ^A T _E X report assembly; Tables 1–8; bibliography; report narrative and English.	Matplotlib, L ^A T _E X; <code>results/*.npz</code> schema as figure interface.
Rong Yin (2363466)	Rong Yin	<code>code/validation/</code> package: independent solver set, reference oracle chain, PASS/FAIL test suite, edge-case and convergence studies, <code>run_all.py</code> .	SciPy <code>solve_ivp</code> (Radau/BDF/LSODA) for reference construction only; interface tested by the dual-tolerance repeat and cross-family comparison.

APPENDIX B — INDIVIDUAL CONTRIBUTION STATEMENTS

Each statement below follows the report template’s four-question format and is signed by its author; the claims are traceable to the team’s git history.

B1. Yi Zhu (2361666) — A. Project Manager

1. My specific contributions to this project.

(1) I ran the weekly team meetings and made sure every member had clear weekly work. I created the git repository and the folder structure, and recorded the team’s challenges, results and milestones on the Learning Mall every week.

(2) Following the project requirements for the Robertson problem, I divided the project into four parts and assigned them to the team members. For the mathematical-theory and algorithm-implementation parts I prepared two task-specification PDFs describing what each part involved and how to carry it out.

(3) After the first version of the report was released, I used AI to compare it against the project requirements to identify errors and missing parts (Appendix A, Items 1–2). I proposed the corresponding modifications and independently conducted the matched-error comparison tests for IE and EE reported in Section 4.4.

2. The hardest conceptual obstacle I encountered. Understanding the Robertson problem well enough to divide it into well-defined tasks and to judge whether members’ numerical results were trustworthy — in particular, what “matched endpoint error” must mean before an EE–IE runtime comparison is fair. I overcame it by studying the problem pack and the derivation document with the Mathematical Theory owner, and by designing and running the matched-error comparison tests of Section 4.4 myself.

3. One piece of AI-generated output my team used, and what I changed. The AI-assisted comparison of the first report draft against the project brief, which proposed adding a matched-error benchmark and a reference-refinement check (Appendix A, Items 1–2). I did not accept the proposed conclusions: I re-ran both experiments independently, and only the re-measured numbers (runtime ratios 5.16–5.25; maximum endpoint change 4.950×10^{-12}) entered the report.

4. One thing I still do not fully understand. How to use AI to learn this module without skipping the process: AI gives results conveniently, but sometimes we knew the result without knowing the reason exactly.

Signature: _____

朱熠

Date: _____

2026.9.29

B2. Zhengdao Song (2361511) — B. Mathematical Theory

1. My specific contributions to this project.

(1) Derived the Robertson system properties: proved the conservation law $y_1 + y_2 + y_3 = 1$ by showing all reaction terms cancel, and established non-negativity as a diagnostic independent of conservation.

(2) Derived EE, classical RK4 and IE from first principles via Taylor expansions, each with local truncation and global error analysis; derived the Newton iteration for IE, including the residual definition, the linearisation, and the rationale for a linear solve rather than an explicit matrix inverse, with tolerance-selection guidelines.

(3) Established the convergence-testing framework: the observed-order formula for paired step sizes, the asymptotic-regime conditions, and the requirement of four consecutively halved step sizes.

(4) Derived the absolute-stability functions of all three methods on the Dahlquist equation and proved IE A-stable; derived the 3×3 Robertson Jacobian element by element, verified the zero column sums and the structural zero eigenvalue, and developed the 2×2 reduced Jacobian used for the stiffness ratio.

(5) Derived the EE stability limit $h < 2/|\lambda|_{\max} \approx 5.9 \times 10^{-4}$ as a local linearisation estimate, and defined the four unified diagnostic quantities together with the matched-accuracy comparison methodology and the Newton-tolerance sweep design.

2. The hardest conceptual obstacle I encountered. The structural zero eigenvalue of the 3×3 Robertson Jacobian: a direct eigenvalue computation mixes the exact zero with the two physical eigenvalues, so the stiffness ratio is awkward to define and compute robustly. I overcame it by deriving the 2×2 reduced Jacobian, which yields the two non-zero eigenvalues directly and is what the report’s stiffness ratios are based on (Section 1.2).

3. One piece of AI-generated output my team used, and what I changed. AI-assisted derivations claimed the Jacobian’s structural zero eigenvalue from the conservation law and the A-stability of IE (Appendix A, Item 3). I accepted both only after re-deriving them by hand — the column sums element by element, and $|1/(1-z)| < 1$ for $\text{Re}(z) < 0$ directly — and I changed the stiffness analysis to use the 2×2 reduced Jacobian instead of the full 3×3 eigenvalue computation.

4. One thing I still do not fully understand. Whether the frozen-Jacobian (local linearisation) stability picture can be turned into a rigorous nonlinear stability statement for this system; the limits I derived are local estimates confirmed by step-size sweeps, not nonlinear proofs (Sections 2.3 and 4.2).

Signature: 宋政道 Date: 2026.9.29

B3. Fangchi Shi (2359837) — C. Algorithm Implementation

1. My specific contributions to this project.

(1) Organised the production solver framework around a general right-hand-side interface $f(t, y)$ rather than hard-coding the Robertson system, so the same implementations could be tested on simpler ODEs first.

(2) Implemented the fixed-step explicit methods (EE and classical RK4), including the final shortened step so the solver lands exactly on the final time.

(3) Implemented implicit Euler with Newton iteration: the nonlinear residual $G(z) = z - y_n - hf(t_{n+1}, z)$, the analytical-Jacobian system $I - hJ$, and convergence controls on the Newton tolerance and iteration cap.

(4) Implemented the adaptive step-doubling controller (`adaptive.py`): one full step versus two half steps for the local error estimate, accept/reject logic, the tolerance-driven step update, and minimum/maximum step limits.

(5) Connected the Robertson right-hand side and its analytical Jacobian to the general solvers (`model.py`), exposed accepted/rejected-step and Newton diagnostics for the Testing & Validation role, integrated the methods into the end-to-end `run_all.py` workflow, and fitted the log-log convergence data for the individual challenge (slope 0.986 ± 0.004 with a 95% confidence band, $R^2 \approx 0.99995$).

2. The hardest conceptual obstacle I encountered. Implicit Euler with Newton iteration: a step can be accepted only after the nonlinear solve converges, so the time-stepping logic and the Newton convergence controls cannot be developed separately. I overcame it by formulating the residual $G(z) = z - y_n - hf(t_{n+1}, z)$ with the analytical system $I - hJ$ and adding explicit controls on the Newton tolerance and the iteration cap (`methods.py`).

3. One piece of AI-generated output my team used, and what I changed. The AI-designed matched-error benchmark protocol (Appendix A, Item 1) ran on my production solvers. What I changed: the protocol needed diagnostics the solvers did not expose, so I added the accepted/rejected-step and Newton-iteration diagnostics while leaving the stepping code itself untouched, so that the benchmark measured exactly the code described in Section 3.

4. One thing I still do not fully understand. Why local error control by step doubling does not guarantee the global error: the adaptive runs keep the local estimate under tolerance while the endpoint error decreases much more slowly (Section 4.3), and I still do not fully understand the mechanism behind that gap.

Signature: 施子强 Date: 2026.9.29

B4. Xiangyu Zhang (2361653) — D. Visualization & Report

1. My specific contributions to this project.

(1) Owned the figure pipeline `make_figures.py` and produced all report figures (Figures 1–10) from `results/*.npz`, applying the visualization checklist: logarithmic axes where required, labelled axes and units, readable legends, no rainbow colormaps, and captions stating what is shown and why it matters.

(2) Maintained the report $\text{\LaTeX}$ source, equation numbering, cross-references, Tables 1–8 and the 13-item bibliography; compiled the final PDF and checked consistency of notation and numbering across the abstract and Sections 1.3, 3.2, 4 and 5.

(3) Drafted or edited the abstract, Section 1.3, the code-structure description in Section 3.2, the opening of Section 4 and the conclusions, integrating the mathematical, algorithmic and validation material into a single experiments–evidence–interpretation narrative.

(4) Cross-checked every caption and in-text number against `results/*.npz` and the tables (endpoint $y(40)$, stiffness ratios $2996 / 5423 / 1.5840 \times 10^5$, the $y_2(40)$ cross-check values), flagged where the text needed “consistency check, not a true-error bound”, and coordinated with Algorithm Implementation so `run_all.py` regenerates all figures and tables. I did not implement the solvers and do not claim their derivations as my own.

2. The hardest conceptual obstacle I encountered. Knowing which claims a figure may support: Figure 4’s stability regions with $h\lambda$ trajectories are a local frozen-Jacobian diagnostic, not a nonlinear stability proof, and Figure 8 shows that mass conservation does not certify Newton convergence. I overcame it by checking every caption against `results/*.npz` and the tables, and confirming the wording with the Testing & Validation owner before fixing the final captions.

3. One piece of AI-generated output my team used, and what I changed. AI-drafted figure captions and $\text{\LaTeX}$

styling (Appendix A, A1 and the routine-use summary). What I changed: I rewrote the captions that overclaimed — adding the “local linear diagnostic” wording and the “consistency check, not a true-error bound” qualifiers — after verifying every quoted number against the tables and `results/*.npz`.

4. One thing I still do not fully understand. The measurement-floor problem: how to present convergence honestly when errors are measured against a numerical reference whose own finite accuracy hides the method’s order near the floor, as in the RK4 case.

Signature: 张祥东 Date: 2026.9.29

B5. Rong Yin (2363466) — E. Testing & Validation

1. My specific contributions to this project.

(1) Built the reference-solution credibility chain (`code/validation/reference.py`): Radau repeated with tolerances tightened 100-fold on the protocol grid, cross-validated against LSODA and BDF, sanity-checked against the problem-pack values; the report quotes only digits confirmed by all checks.

(2) Implemented an independent solver set (`solvers.py`: EE, RK4, IE with Newton, step-doubling controller, cost ledger), deliberately separate from the production code so validation never depends on the code it checks.

(3) Wrote the eight-test PASS/FAIL suite on the validation branch (`validation_tests.py`, 8/8 PASS): end-point versus oracle, conservation, non-negativity, order checks, Newton residual trace, eigenvalue check values, oracle self-agreement.

(4) Ran the edge-case experiments (`edge_cases.py`): EE/RK4 sweeps confirming the frozen-Jacobian limits $5.9 \times 10^{-4} / 8.2 \times 10^{-4}$, including the honest “stable but negative” case at $h = 6 \times 10^{-4}$; the Newton-tolerance sweep, which falsified the generic assumption that the conservation defect scales with the tolerance and led to the exact invariant-preservation argument ($e^T J = 0$) used in Section 4.4; and the adaptive-tolerance sweep showing local control is not a global guarantee.

(5) Ran the convergence studies with fitted slopes and 95% bands, documenting the RK4 measurement-floor limitation; ran the pre-submission final check (`final_check.py`), including the $y_2(40)$ five-level cross-check and the report-number traceability table corroborating Section 4; verified `run_all.py` end-to-end from a clean state.

2. The hardest conceptual obstacle I encountered. The Newton-tolerance sweep contradicted the generic assumption that IE’s mass-conservation defect scales with the tolerance — the defect stayed at round-off for every tolerance tested. Turning that surprise into an explanation required deriving the reason: $e^T J = 0$ implies every Newton iterate preserves the linear invariant exactly; that argument is now used in Section 4.4.

3. One piece of AI-generated output my team used, and what I changed. The AI-generated Radau reference oracle, initially treated as ground truth (Appendix A, Item 2). What I changed: I did not quote its digits directly — I repeated Radau with the tolerances tightened 100-fold, cross-checked against BDF and LSODA, and the report quotes only the digits confirmed by all checks.

4. One thing I still do not fully understand. When adaptive stepping actually pays off for stiff problems: at the matched target error $\approx 7 \times 10^{-5}$ my step-doubling controller cost about $1.6\times$ the function evaluations of fixed stepping, and I still do not fully understand in which regime adaptivity wins.

Signature: 尹榕 Date: 2026.9.29

APPENDIX C — SUPPLEMENTARY FIGURES AND DATA

Additional tables and data that support Sections 4.1 and 4.4 of the main report but would interrupt the narrative there. All values are taken from the same runs reported in the main text; a dash marks a quantity not separately recorded for that row.

C.1 Newton-tolerance sweep at $h = 0.05$

Table 1 gives the values behind the right panel of Figure 8: implicit Euler at $h = 0.05$ with the Newton residual tolerance τ_N varied, all other settings at their production values.

C.2 Matched-error benchmark protocol (supplementary to Figure 10)

The supplementary matched-error benchmark of Section 4.4 compares EE and IE at four accuracy targets (pairs P1–P4). Table 2 records the protocol and environment; the measured runtimes are plotted in Figure 10.

Table 1: Newton-tolerance sweep for implicit Euler at $h = 0.05$ (supports Figure 8, right panel). The error plateau at 592.6 is the pure time-discretisation error at this step size: once $\tau_N \leq 10^{-6}$ the algebraic error no longer affects the endpoint, while the mean iteration count grows from 1.23 to 2.22. Mass is conserved to round-off in every converged row — including the badly wrong $\tau_N = 10^{-4}$ run, whose mass defect is 3.77×10^{-15} — which is why conservation alone is not an accuracy check (Section 4.4).

τ_N	endpoint error E	$y_2(40)$	mean Newton its.	outcome
10^{-2}	non-finite	—	—	iteration diverges
10^{-4}	8.47×10^6	-2.5663×10^{-6}	—	converged, but inaccurate and negative
10^{-6}	592.6	—	1.23	error plateau reached
10^{-10}	592.6	—	—	production choice
10^{-13}	592.6	—	2.22	tighter, no accuracy gain

Table 2: Matched-error benchmark: protocol and environment. The four target errors E^* are the measured EE weighted endpoint errors at $N = 80,000, 160,000, 320,000$ and $640,000$ steps; an IE run is accepted as matched when $0.95 E^* \leq E_{IE} \leq E^*$. At the strictest target ($E^* = 0.7423$, $N = 640,000$ for both methods) the runtimes are EE 0.98 s and IE 5.07 s; the IE/EE runtime ratio is 5.16–5.25 across the four pairs. These timings belong to this benchmark and should not be combined with the Figure 9 series.

Item	Setting
Error metric	weighted endpoint error at $t = 40$, fixed weights $10^{-8} + 10^{-6} y_i^{\text{ref}} $
IE nonlinear solve	Newton residual tolerance 10^{-13} , at least one correction per step
Timing	one warm-up run, then five timed runs, minimum reported
Included	solver initialisation, RHS and Jacobian evaluations, linear solves, output sampling, trajectory diagnostics
Excluded	reference construction, parameter selection, plotting, file I/O
Environment	Python 3.12.14, NumPy 2.2.6, SciPy 1.15.3; macOS 26.6.2, arm64

C.3 Reproducibility record

- `python code/run_all.py` regenerates every number, table and figure of the report (Tables 4–8, Figures 1–10) from scratch in a few minutes; the run was verified end-to-end from a clean state.
- The production validation suite contains ten PASS/FAIL tests, all passing (Section 4.1); the independent validation branch (`code/validation/`) re-implements the solvers and the reference oracle and runs its own eight-test suite (Section 4.1), so that checking never depends on the code being checked.
- The reference oracle chain behind Table 4 and Figure 2: Radau at $(\text{rtol}, \text{atol}) = (10^{-10}, 10^{-12})$ repeated at $(10^{-12}, 10^{-14})$, compared at $t = 0$ plus 241 logarithmically spaced output times in $[10^{-8}, 40]$ (solver internal steps are independent of these output times), with an independent BDF run as a cross-family check.